4.2、顺序表类模板

```
template <typename T,int size>class seqlist
{
    T slist[size];   //存放顺序表的数组
    int Maxsize;   //最大可容纳项数
    int last;        //已存表项的最后位置
public:
    seqlist() { last=-1; Maxsize=size; }    //初始化为空表
    int Length() const{return last+1;} //计算表长度
    int Find(const T & x)const;       // 寻找x在表中位置（返回下标）
    bool IsIn(const T & x) const;         //判断x是否在表中
    bool Insert(const T & x,int i);        //x插入到列表中第i个位置处（下标）
    bool Remove(const T & x);           //删除x
    int Next(const T & x) const;          //寻找x的后继位置
    int Prior(const T & x) const;         //寻找x的前驱位置
    bool IsEmpty() const{return last==-1;}                  //判断表是否空
    bool IsFull() const{return last==Maxsize -1;}         //判断表是否满
    const T& Get(int i) const {return i<0||i>last?NULL:slist[i];} //取第i个元素
   T& operator[](int i);   //重载下标运算符[]
};
```

计算表长度
判断某变量或对象x是否在表中
寻找某变量或对象x在表中的位置
插入
删除
寻找x的后继和前驱
判断表为空或者满
取第i个元素的值

主程序检验

4.2、顺序表类模板

◆ 寻找x在表中位置，返回下标

```
template <typename T,int size>
int seqlist<T,size>::Find(const T & x) const
{
  int i=0;
  while( i<=last && slist[i]!=x) i++;  //顺序查找是否有x
  if (i>last) return -1; //未找到，返回-1
  else return i; //找到，返回下标
}
```



4.2、顺序表类模板

◆ 判断x是否在表中

```
template <typename T,int size>
bool seqlist<T,size>::IsIn(constT & x) const
{
        int i=0;
        bool found=0;
        while(i<=last && !found)
             if (slist[i]!=x) i++;
            else found=1; //找到
        return found;
}
```



4.2、顺序表类模板

◆ 将X插入表中第i个下标的位置

```
template <typename T,int size>
 bool seqlist<T,size>::Insert(constT & x, int i)
{
        int j;
        if ( i<0 || i>last+1 || last==Maxsize -1)
              return false;
      else
        {
              last++;
              for(j=last;j>i;j-- ) slist[j]=slist[j-1];
              slist[i]=x;
              return true;
         }
}
```

判断插入位置是否合理

从表最后位置向前依次后移，空出指定位置来



4.2、顺序表类模板

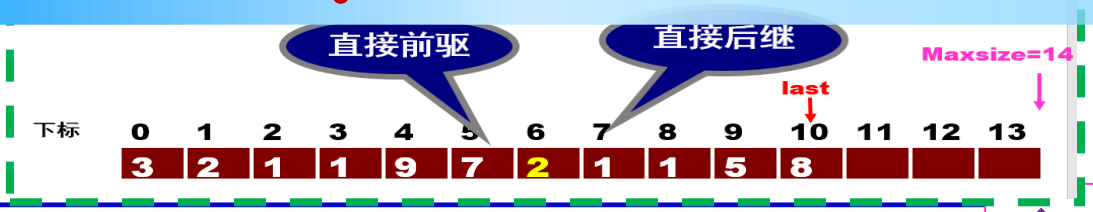
```
template <typename T,int size>
 bool seqlist<T,size>::Remove(constT & x)
{
        int i,j;
        i=Find(x);
       if(i>=0)
        {
              last--;
              for( j=i;j<=last;j++ )
                    slist[j]=slist[j+1];
          return true;
         }
        return false; //表中不存在x
}
```

先找x在哪个位置

◆ 删除一个元素



4.2、顺序表类模板



◆ 查找X元素的后继

```
template <typename T,int size>
int seqlist<T,size>::Next(constT & x) const
{
    int i= Find(x);
    if(i>=0 && i<last)  return i+1; //x后继位置
    else return -1; //x不在表中，或在表末尾
}
```

◆ 查找X元素的前驱

```
template <typename T,int size>
int seqlist<T,size>::Prior(constT & x) const
{
    int i= Find(x);
    if( i>0 && i<=last )  return i-1; //x前驱的位置
    else return -1;
}
```

4.2、顺序表类模板

```
template <typename T,int size> T& seqlist<T,size>::operator[](int i)
{
    if(i>last+1||i<0||i>=Maxsize)
    {
        cout<<"下标出界！ "<<endl;
        exit(1);
    }
    if(i==last+1) last++;
    return slist[i];
}
```

◆ 重载[ ]运算符



4.2、顺序表类模板应用示例

```
class seqlist
{      int slist[100], Maxsize, last;
     public: <略>
};
```

```
int main()
{
  seqlist <int,100> seqlisti;
  int i,j,k,a[10]={2,3,5,7,11,13,17,19,23,29};
  for(j=0;j<10;j++)
   if (!seqlisti.Insert(a[j],j))
    {   cout<<"表太大放不下了!"<<endl;  break; }
  j=seqlisti.Length();
  for(i=0;i<j;i++) cout<<seqlisti.Get(i)<<' ';
  cout << endl ;
  for(j=0;j<10;j++) seqlisti[j]=0;
  for(j=0;j<10;j++) cout<<seqlisti[j]<<' ';
   cout<<endl;
  for(j=0;j<10;j++) seqlisti[j]=a[j];
```

顺序表对象seqlisti的元素为整型

把素数写入

打印素数

采用下标运算符运算

实验能否增加元素

```
seqlisti[10]=31;
for(j=0;j<11;j++) cout<<seqlisti[j]<<' ';
cout<<endl;
```

判断元素是否存在

```
k=7;
 if (seqlisti.IsIn(k)) cout<<"素数7在顺序表中"<< endl;
 else cout <<"素数7不在顺序表中"<<endl;
```

```
k=17;
 if (seqlisti.Remove (k)) cout<<"删除素数17"<<endl;
 else cout<<"找不到素数17，无法删除";
```

删除元素

```
j=seqlisti.Length( ) ;
for (i=0;i<j;i++) cout<<seqlisti.Get(i) <<' ';
  //打印剩下的素数
 cout<<endl;
```




```
if (seqlisti.Insert(k,j-1))
{
    j=seqlisti.Length ( );
    for (i=0;i<j;i++)  cout<<seqlisti.Get(i) <<' ';
      cout<<endl;
 }
```

插入一个元素，并打印

```
cout<<"打印17后一个素数:
“<<seqlisti.Get(seqlisti.Next(k))<<endl;
cout<<"打印17前一个素数:  “
   <<seqlisti.Get(seqlisti.Prior(k))<<endl;
cout<<"素数17在表中位置（下标）为:  “
   <<seqlisti.Find(k)<<endl;
```

```
if(seqlisti.IsEmpty( ))  cout<<"表是空的"<<endl;
 else cout<<"表不空"<<endl;
if (seqlisti.IsFull())  cout<<"表是满的"<<endl;
  else cout<<"表也不满"<<endl;
if (seqlisti.IsIn(k))  cout<<"素数17在表中"<<endl;
```

清空

```
seqlist.clear();
```

{last=-1;}

?清空代表什么？

```
if(seqlisti.IsEmpty( ))  cout<<"表是空的"<<endl;
else cout<<"表不空"<<endl;
  return 0;
}
```




查找与排序

5、查找与排序

查找(search)：

- 在数据集合中寻找满足条件的数据对象，找到后进一步给出对象的具体信息，在数据库技术中称为检索。

排序(sorting)：

- 例如字典：字典中的词条是按序存放的，我们才能按字母顺序找到要查的字。

5.1、查找

查找按key word进行。可以唯一地把资料区分出来的数据称为主关键字。

如学生的资料(结构体变量):

```
struct  student
{
    int  id ;                  //学号：主关键字
    char name[20];     // 姓名
    char sex;                 // 性别
    int  age;                 // 年龄
    char address[60]; //家庭地址
    float  eng, phy,math , electron; //各科成绩
};
```

5.1、查找

常用查找方法

- **顺序查找：**

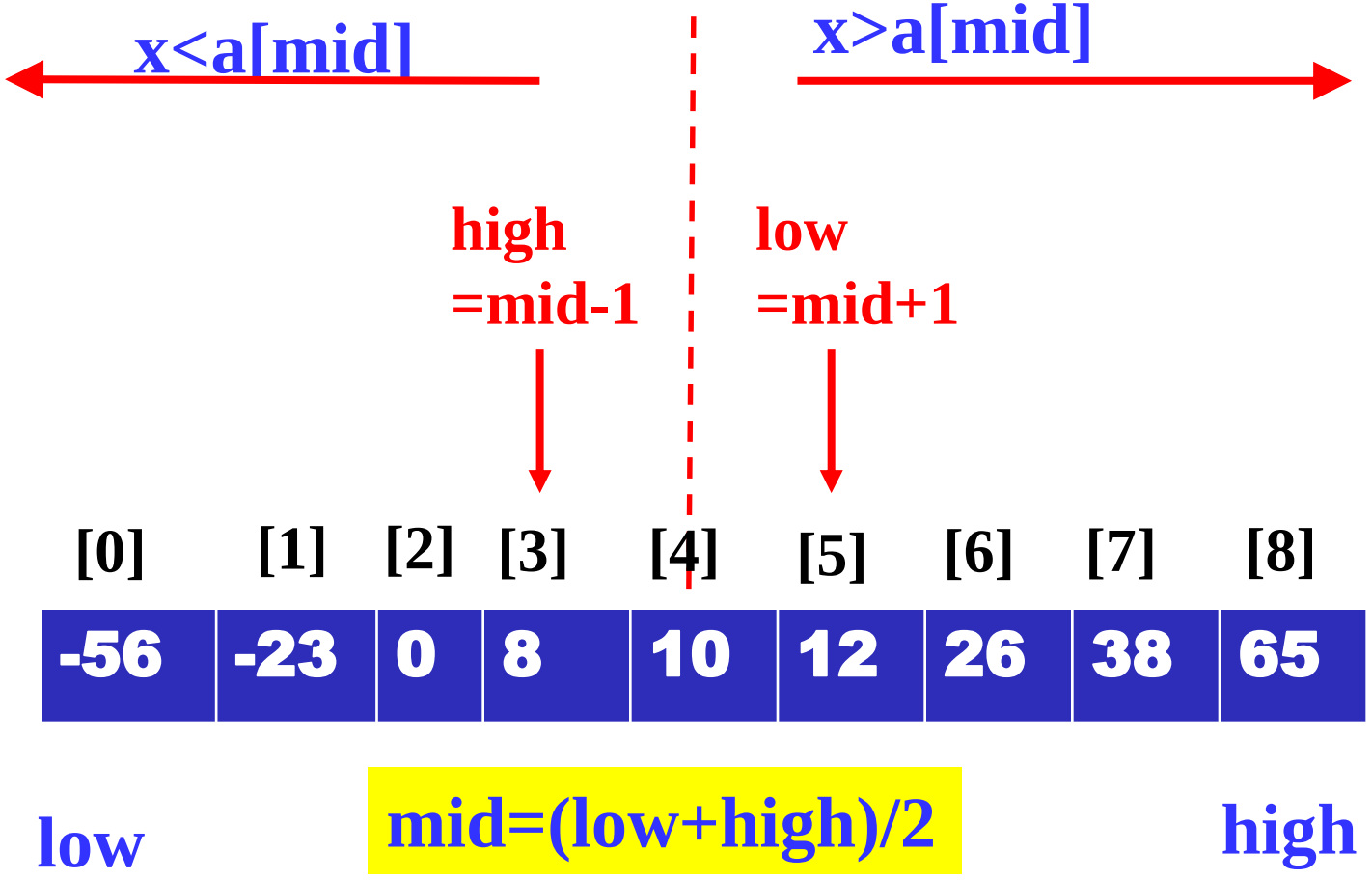
从第一个元素开始，顺序查找直到找到或查到最后一个元素为止。

- **对半查找（折半查找）：**

针对**有序数据**的查找。若关键字小的排在前面，称为升序排列，反之为降序排列。

5.1、查找

折半查找


x<a[mid]
x>a[mid]
high
=mid-1
low
=mid+1
[0]
[1]
[2]
[3]
[4]
[5]
[6]
[7]
[8]
a
-56
-23
0
8
10
12
26
38
65
low
mid=(low+high)/2
high

5.1、查找-折半查找

递归折半查找算法（类模板成员函数）

```
template <typename T,int size>class Orderedlist
{
    int maxsize;
    int last;
    T slist[size];
  public:
    Orderedlist(){last=-1;maxsize=size;}
   int Binarysearch(const T & x,const int low, const int high);
   bool Insert(T & elem,int i);
   void print();
      // 无关成员函数省略
};
```

有序表类模板定义

5.1、查找-折半查找

递归折半查找算法（类模板成员函数）

```
template <typename T, int size>
int Orderedlist<T,size>::Binarysearch
(const T & x,const int low,const int high)  // x为定值
{
    int mid=-1;
    if (low<=high)
    {
        mid=(low+high)/2;
        if(slist[mid]<x) mid=  Binarysearch(x,mid+1,high);
    else if(x<slist[mid]) mid= Binarysearch(x,low，mid-1);
     }
     return mid;
}
```

5.1、查找-折半查找

```
template <typename T,int size>
bool Orderedlist<T,size>::Insert(T & elem ,int i)
{
        int j;
        if (i<0||i>last+1||last==maxsize-1)
         return false;
        else
        {
                last++;
                for (j=last;j>i;j--) slist[j]=slist[j-1];
                slist[i]=elem;
                return true;
        }
}
```

有序表类模板Insert成员函数

5.1、查找-折半查找

```
template <typename T,int size>
 void Orderedlist<T,size>::print()
{
        int i;
        for(i=0;i<=last;i++)
        {
                slist[i].show();
                if(i%5==4) cout<<endl;
        }
        cout<<endl;
}
```

有序表类模板print成员函数

5.1、查找-折半查找

类Elemen定义：

```
class Element
{
        int  key;       // 其他域省略
  public:
        bool operator<(Element ele)
                {return key<ele.key;}
        void putkey(int k){key=k;}
        void show(){cout<<key<<'\t';}
};
```

比较运算符重载，元素的比较是元素关键字的比较

5.1、查找-折半查找

```
int main(){
        const int h=19;int i,k=37;
        Orderedlist<Element,100> ordlist;
        int a[h]={67,61,59,53,47,43,41,37,31,29,23,
                  19,17,13,11,7,5,3,2}; //降序
        Element n[h], elem;
        for(i=0;i<h;i++)  n[i].putkey(a[i]);
        for(i=0;i<h;i++)  ordlist.Insert(n[i],0);
        elem.putkey(k);
        ordlist.print();
        i=ordlist.Binarysearch(elem,0,h-1);
        cout<<"整数"<<k<<"在表中位置（下标）： "<<i<<endl;
        return 0;
}
```

有序表，基本元素为类Eelement，最大长度为100

对象数组

始终在0号元素插入，建立升序顺序表

5.1、查找-折半查找

迭代折半查找算法（类模板成员函数）

```
template <typename T,int size >
 int  Orderedlist<T,size>::BinarySearch2(const T & x)
{
   int high=last ,low=0,mid;
   if (last==-1) return -1;
   while(low<=high)
   {
      mid=(low+high)/2;
      if (x==slist[mid]) return mid;
      else   if (x<slist[mid])       high=mid-1;
      else         low=mid+1;
    }
  return -1;
}
```

5.2、查找-索引查找

索引：就象一种目录（地址记录），找到标题对应的页号（具体地址），立即可以找到对应的内容。

可利用指针数组实现。

一维指针数组



学生记录

学号	姓名	性别	年龄
04002808	张伟	男	18
04002804	姚婕	女	17
04002802	王茜	女	18
04002807	朱明	女	18
04002809	沈俊	男	17
04002806	况钟	女	17
04002801	程毅	男	18
04002803	李文	男	19
04002805	胡凤	女	19



表比较大，非有序表，不能用折半查找，但是顺序查找比较费时，怎么办？

考虑一种情况：如果整个表中的元素未必有序，但若划分为若干块后，每一块中的所有元素均小于（或大于）其后面块中的所有元素。

9	4	14	5	45	26	22	34	65	58	48	60

可以建立一个索引表，索引表中为每一块都设置一索引项。

王文博 2026

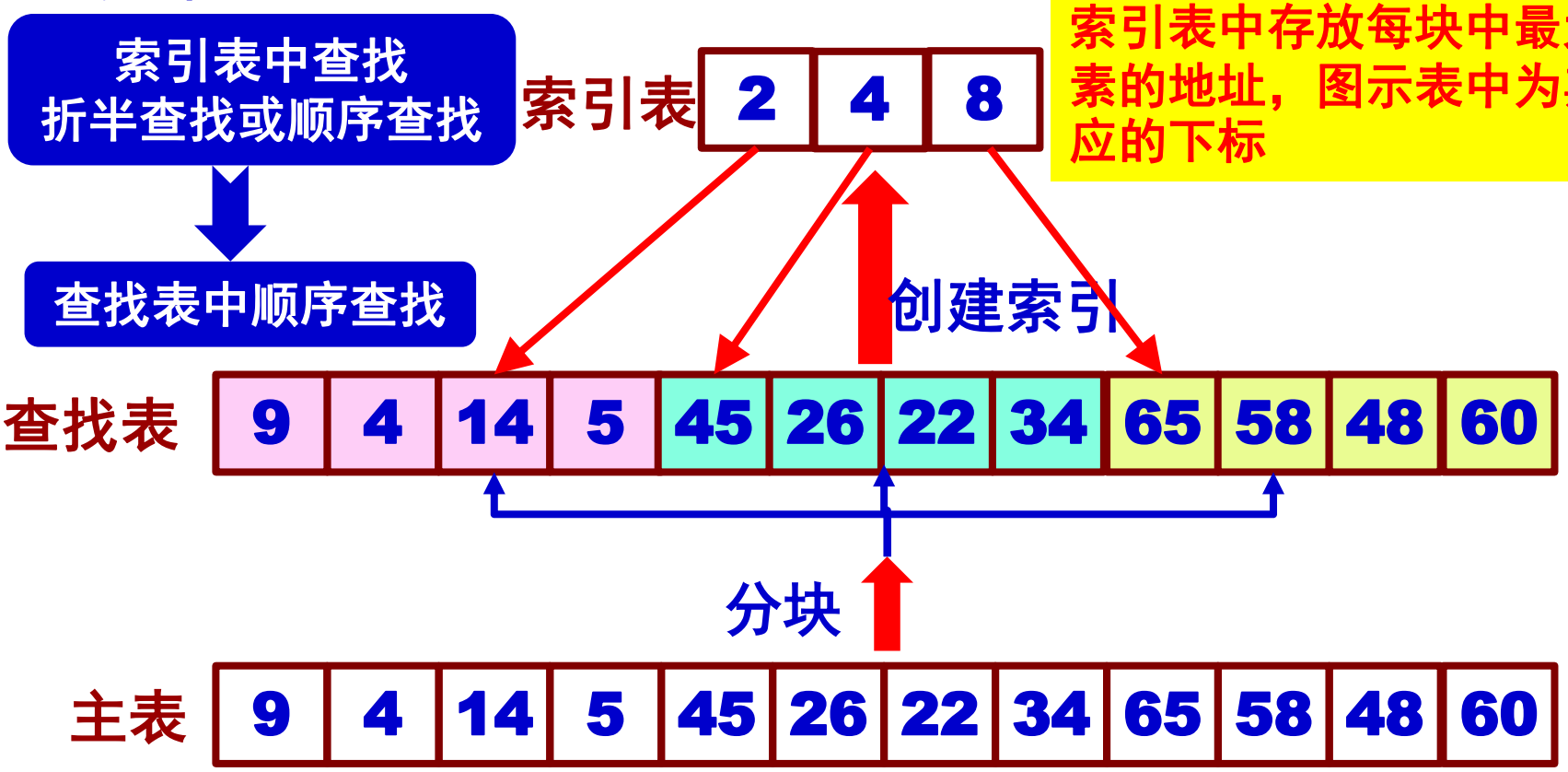
5.1、查找-分块索引查找

查找过程：

索引表中查找
折半查找或顺序查找

查找表中顺序查找

索引表中存放每块中最大值元素的地址，图示表中为其中对应的下标



分块查找是一种介于顺序查找和二分查找之间的一种查找方法

哈希查找

哈希（hash）查找：

在记录的存储位置和记录的 **key** 之间建立一个确定的映射 **f()**，使得每个 **key** 对应一个存储位置 **f(key)**。



感兴趣的同学可以自行查找资料了解

5.2、排序

常用的排序法（1）

- 插入排序(InsertSort)
- 交换排序(冒泡BubbleSort)
- 选择排序(SelectSort)

5.2、排序-插入排序

直接插入排序(升序): 当插入第i(i>=1)个元素时,前面的元素0,1,..., i-1个元素已经排好序, 将第i个元素的关键字与i-1, i-2,...,的关键字顺序进行比较,找到第一个比它小的,则第i个元素插到该元素之后。

i	0	1	2	3	4	5	6	temp
初始序列	[8]	6	7	9	4	5	2	6

5.2、排序-插入排序算法实现1

```
template <typename T,int size>
class Orderedlist
{   int maxsize;
    int last;
  T slist[size];
  略
};
```

```
//Orderedlist<T,size>类的成员函数

template <typename T,int size>
void Orderedlist <T,size> :: InsertSort()
{
        T temp;
        int i,j;
        for (i=1;i<=last;i++)
        {
                temp= slist[i];
                j=i;
                while (j>0&&  temp<slist[j-1])
                {
                        slist[j]=slist[j-1];
                        j--;
                }
               slist[j]=temp;
        }
 }
```

要进行last=N-1轮循环

找到要插入的位置

把插入位置腾出

把数据插入

5.2、排序-插入排序



前面直接插入排序的算法中，插入第i(i>=1)个元素时，需将第i个元素的关键字与i-1, i-2,...,的关键字顺序进行比较,找到第一个比它小的

```
j=i;
while ( j>0&& temp<slist[j-1])
{
      ....
      j--;
}
```

有没有更快的算法？

5.2、排序-插入排序算法实现2

```
template <typename T,int size>
void Orderedlist <T,size> :: BinaryInsertSort()
{       T temp;
        int low,high,mid,i,j;
        for (i=1;i<=last;i++){
            temp=slist[i];
            low=0;
            high=i-1;
            while (low<=high){
                  mid=(low+high)/2;
                  if(temp<slist[mid])  high=mid-1;
                  else  low=mid+1;
                  }
        for(j=i-1;j>=low;j--) slist[j+1]=slist[j];
        slist[low]=temp;
        }
}
```

折半查找

找到要插入的位置

把插入位置腾出

把数据插入

注意与折半查找的不同

对半插入排序算法

5.2、排序

冒泡排序

5.2、排序-交换排序（冒泡排序）

原始数据排列



冒泡排序



需要交换　需要交换　需要交换

当前需要冒泡的位置

从后向前冒泡

如何判断排序完成？

N个元素，一轮冒泡需要比较N-1次

至多要进行N-1轮冒泡，但是有可能提前完成。

5.2、排序-交换排序（冒泡）算法实现

```
template <typename T,int size>
void Orderedlist <T,size>::BubbleSort(){
        bool noswap;
        int i,j;
        T temp;
        for (i=0;i<last;i++){
                noswap=true;
        for(j=last;j>i;j--){
                if(slist[j]<slist[j-1]){
                        temp=slist[j];
                        slist[j]=slist[j-1];
                        slist[j-1]=temp;
                        noswap=false; } }
        if(noswap) break;
    }
}
```

flag

未交换为真

从下往上冒泡

两个数据交换

无交换，终止算法

5.2、排序

选择排序

5.2、排序-选择排序

i	a[i]
0	45
1	64
2	24
3	72
4	18
5	8

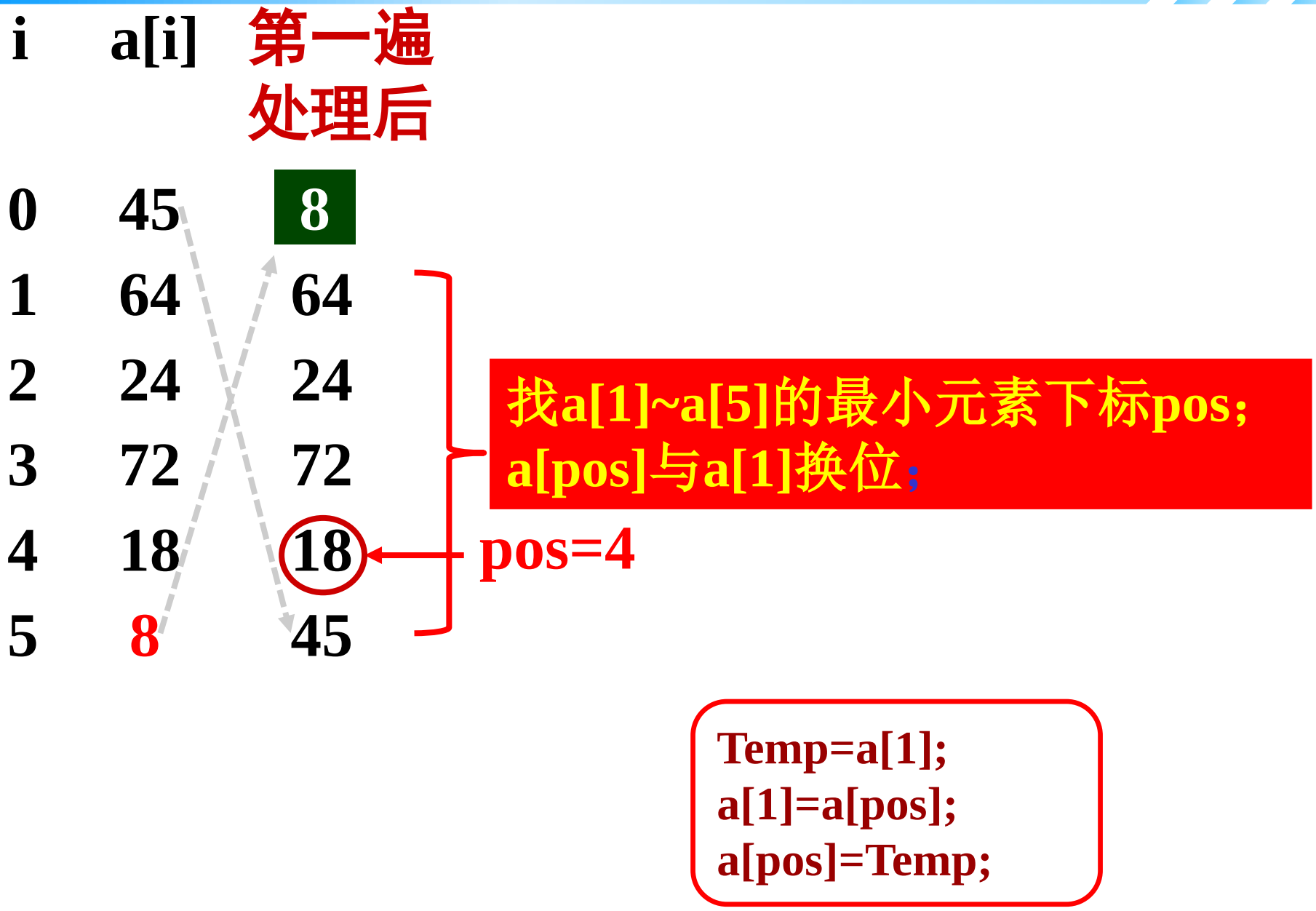
pos=5

找a[0]~a[5]的最小元素下标pos;

a[pos]与a[0]换位;

```
Temp=a[0];
a[0]=a[pos];
a[pos]=Temp;
```

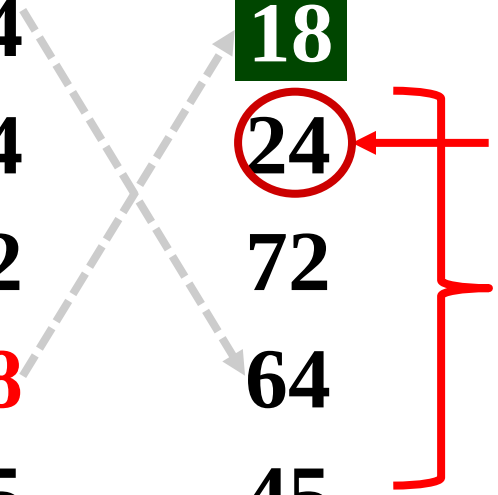
5.2、排序-选择排序




```
Temp=a[1];
a[1]=a[pos];
a[pos]=Temp;
```

5.2、排序-选择排序

i	a[i]	第一遍处理后	第二遍处理后
0	45	8	8
1	64	64	18
2	24	24	24
3	72	72	72
4	18	18	64
5	8	45	45



pos=2

找a[2]~a[5]的最小元素下标pos;

pos=2,无需交换

5.2、排序-选择排序

i	a[i]	第一遍处理后	第二遍处理后	第三遍处理后
0	45	8	8	8
1	64	64	18	18
2	24	24	24	24
3	72	72	72	72
4	18	18	64	64
5	8	45	45	45

找a[3]~a[5]的最小元素下标pos；

pos=5

a[pos]与a[3]换位；

5.2、排序-选择排序

i	a[i]	第一遍处理后	第二遍处理后	第三遍处理后	第四遍处理后
0	45	8	8	8	8
1	64	64	18	18	18
2	24	24	24	24	24
3	72	72	72	72	45
4	18	18	64	64	64
5	8	45	45	45	72

pos=4

找最小元素下标pos；

pos=4 无需换位；

5.2、排序-选择排序

i	a[i]	第一遍处理后	第二遍处理后	第三遍处理后	第四遍处理后	第五遍处理后
0	45	8	8	8	8	8
1	64	64	18	18	18	18
2	24	24	24	24	24	24
3	72	72	72	72	45	45
4	18	18	64	64	64	64
5	8	45	45	45	72	72

5.2、排序-选择排序算法实现

```
template <typename T>
void Orderedlist<T>::SelectSort() {
    int i,j,k;
    T tempmin;
    for (i=0;i<last;i++) {
            k=i;
            tempmin=slist[i];
            for (j=i;j<=last;j++)
              if (slist[j]<tempmin)
              {       k=j;    tempmin=slist[j];    }
            if ( k!=i ) {
                    tempmin=slist[i];
                    slist[i]=slist[k];
                    slist[k]=tempmin;
            }
    }
}
```

要进行N-1轮循环

在i~last之间寻找找最小值

判断最小值下标是否是i，否则将其交换到i下标中